NIVEL 1

EXERCICI 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

He creado la base de datos y las tablas con los códigos proporcionados en la plataforma. Al insertar los datos tuve un pequeño inconveniente con la tabla transaction, al ser una palabra reservada por Mysql no me importaba los datos por lo que tuve que hacer un replace en vez de transaction puse la palabra entre backticks (`).

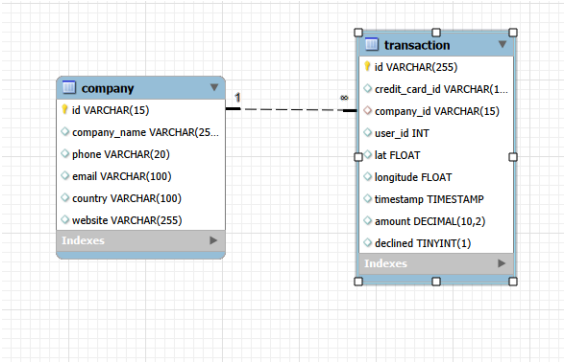
```
-- Base de datos
CREATE DATABASE IF NOT EXISTS transactions;
USE transactions;

-- Tabla company
CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(20),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255)
);
```

```
CREATE TABLE IF NOT EXISTS `transaction` (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15) REFERENCES credit_card(id),
    company_id VARCHAR(20),
    user_id INT REFERENCES user(id),
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    FOREIGN KEY (company_id) REFERENCES company(id)
```

#	Time	Action	Message	Duration / Fetch
15	11:23:15	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, ...	0 row(s) affected	0.016 sec
16	11:23:15	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY...	0 row(s) affected	0.031 sec

```
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('72546752-C2F6-4E5
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('623640C6-AA7E-4B7
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('29063BCD-1886-4E8
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('989054DB-E9E1-40B
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('77338CDA-5EBF-41A
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('DF27ABD1-13A2-4AE
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('5A980850-2921-419
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('FBC7857A-E4BE-497
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('16C38D53-1A6C-426
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('3D39E00C-1F9D-42A
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('E183116A-B83A-412
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('65FF0A2A-3141-437
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('D7AE2C20-8119-4DC
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('C312BA55-0FCC-462
INSERT INTO `transaction` (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('3EFB1984-E1FC-4F4
```

Estructura del esquema



Es un modelo relacional donde la tabla company es la tabla de dimensión y la de transactions la de hechos.

Tabla company

```
26 •  DESCRIBE company;
27 •  DESCRIBE `transaction`;
```

Result Grid | Filter Rows: | Export: | Wrap C

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
company_name	varchar(255)	YES		NULL	
phone	varchar(15)	YES		NULL	
email	varchar(100)	YES		NULL	
country	varchar(100)	YES		NULL	
website	varchar(255)	YES		NULL	

En esta tabla se guarda la información relacionada con la empresa, contiene datos descriptivos como puede ser el nombre o el país donde está la empresa.

ID ⇒ es el Primary Key, es el identificador único. Tipo Varchar, texto límite de carácteres 15.
company_name ⇒ nombre de la empresa. Tipo Varchar, texto con límite de carácteres 255.
phone ⇒ número de teléfono. Tipo Varchar, texto con límite de carácteres 20.
email ⇒ email. Tipo Varchar, texto con límite de carácteres 100.
country ⇒ país. Tipo Varchar, texto con límite de carácteres 100.
website ⇒ web de la empresa. Tipo Varchar, texto con límite de carácteres 255.

tabla transaction

Field	Type	Null	Key	Default	Extra
id	varchar(255)	NO	PRI	NULL	
credit_card_id	varchar(15)	YES		NULL	
company_id	varchar(20)	YES	MUL	NULL	
user_id	int	YES		NULL	
lat	float	YES		NULL	
longitude	float	YES		NULL	
timestamp	timestamp	YES		NULL	
amount	decimal(10,2)	YES		NULL	
declined	tinyint(1)	YES		NULL	

La tabla guarda información sobre las transacciones, cantidad, día, número de tarjeta.

ID ⇒ Es la Primary Key de esta tabla, cada línea es una operación diferente. Tipo Varchar, texto con límite de carácteres 255.
credit_card_id ⇒ id número de tarjeta
company_id ⇒ Foreign Key
user_id ⇒ identificador usuario. Tipo entero
lat ⇒ latitud. Tipo decimal
longitude ⇒ longitud. Tipo decimal
timestamp ⇒ fecha de la operación. Tipo TimeStamp
amount ⇒ importe de la operación. Tipo decimal con dos decimales y 10 números máximos.
declined ⇒ operación declinada o no. Tipo entero pequeño.

La relación que hay entre las dos tablas es de 1:N, uno a muchos, donde una empresa realiza una operación a la vez y una empresa puede tener muchas operaciones. La serie que le conecta sería la de id en la tabla de company y company_id en la tabla de transactions.

EXERCICI 2

Utilitzant JOIN realitzaràs les següents consultes:

- *Llistat dels països que estan generant vendes.*

En esta consulta he usado el SELECT DISTINCT para que no me dé duplicados, al usar INNER JOIN ya conseguimos los países coincidentes que están en las dos tablas, aparecen los países de aquellas empresas que tienen operaciones en la tabla de transactions.

Output:

#	Time	Action	Message	Duration / Fetch
100108	13:39:52	SELECT DISTINCT c.country AS Paises FROM company c JOIN transaction t ON...	15 row(s) returned	0.016 sec / 0.000 sec
100109	13:41:14	SELECT DISTINCT c.country AS Paises FROM company c JOIN `transaction` t O...	15 row(s) returned	0.016 sec / 0.000 sec

Consulta:

```
SELECT DISTINCT c.country AS Paises
FROM company c
JOIN `transaction` t ON t.company_id = c.id;
```

Resultado:

Paises
Germany
Australia
United States
New Zealand
Norway
United Kingdom
Italy
Belgium
Sweden
Ireland
China
Canada

- *Des de quants països es generen les vendes.*

He usado el operador COUNT() junto a DISTINCT para que cuente todos los países sin duplicados.

Consulta:

```
SELECT COUNT(DISTINCT c.country) AS Paises
FROM company c
JOIN `transaction` t ON t.company_id = c.id;
```

Resultado:

Result Grid

Paises
15

Output:

100062	13:31:05	SELECT COUNT(DISTINCT c.country) AS Paises FROM company c JOIN `transac...	1 row(s) returned

- *Identifica la companyia amb la mitjana més gran de vendes*

Para realizar esta consulta he calculado la media de la serie amount, lo filtré por aquellas empresas donde el valor en la columna declined es 0 o lo que es lo mismo False, la operación no fue rechazada. Agrupado por c.id, el orden de la lista resultante es descendente para que nos ordene de mayor a menor y limitamos a 1 el resultado para que nos dé la empresa con una media superior.

Consulta:

```
SELECT c.id, c.company_name, ROUND(AVG(t.amount),2) AS Media_Ventas
FROM company c
JOIN `transaction` t ON t.company_id = c.id
WHERE t.declined = 0
GROUP BY c.id
ORDER BY Media_Ventas DESC
LIMIT 1;
```

Resultado:

Result Grid | Filter Rows: | Export:

id	company_name	Media_Ventas
b-2222	Ac Fermentum Incorporated	284.91

Output:

Output

Action Output

#	Time	Action	Message	Duration / Fetch
100115	13:57:27	SELECT c.id, c.company_name, AVG(t.amount) AS Media_Ventas FROM compan...	1 row(s) returned	0.234 sec / 0.000 sec

EXERCICI 3

Utilitzant només subconsultes (sense utilitzar JOIN):

- *Mostra totes les transaccions realitzades per empreses d'Alemanya.*

Para realizar el ejercicio he creado un consulta de la tabla trasaction donde se realiza un filtro (WHERE) para ver solo con las transacciones que se han realizado en Alemania.

Consulta:

```
SELECT t.*
FROM `transaction` t
WHERE EXISTS (
    SELECT 1
    FROM company c
    WHERE c.id = t.company_id
    AND country = 'Germany'
);
```

Resultado:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0
00687139-48B2-4FFA-8E73-B20376F04AB4	CcS-4870	b-2222	289	51.1966	10.4669	2019-03-09 19:37:49	524.84	0
0074F4DD-32F1-4827-8758-55896314623A	CcS-8081	b-2222	3500	39.7016	-8.50325	2016-12-26 23:06:57	491.90	0
00AAB9CD-39D6-4DCB-8A1D-13BE73DC90A9	CcS-6797	b-2222	2216	55.7652	-3.76245	2021-04-25 03:06:59	167.15	0
00BE09D4-6920-47D8-ABE8-325E2269829D	CcS-4983	b-2222	402	38.708	-9.12993	2019-02-27 15:25:16	141.66	0

Output:

8	10:39:32	SELECT t.* FROM `transaction` t WHERE EXISTS (SELECT 1 FROM compa...	13291 row(s) returned

- *Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.*

Para realizar la consulta lo que primero hice fue hacer una consulta donde calculaba la media del amount de toda la tabla, después hice otra consulta donde comprobaba que ids tienen un 'amount' mayor que la medía y los devuelve..

Consulta:

```sql
SELECT c.id, c.company_name
FROM company c
WHERE EXISTS (
    SELECT 1
    FROM `transaction` t
    WHERE t.company_id = c.id
    AND t.amount > (
        SELECT AVG(amount)
        FROM `transaction`)
);
```

Resultado:

Result Grid | Filter Rows:

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Neque Industries
b-2230	Fusce Corp.
b-2234	Convallis In Incorporated
b-2238	Ante Iaculis Nec Foundation
b-2242	Donec Ltd

Output:

100065	16:11:17	SELECT c.id, c.company_name FROM company c WHERE c.id IN (SELECT t.co...	100 row(s) returned

- *Eliminarán del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.*

La consulta está planteada que calcula en una subconsulta todas los id que aparecen en la tabla transaction para después en una consulta pedir el nombre de las empresas que no están en esa lista.

Consulta:

```
SELECT c.company_name
FROM company c
WHERE NOT EXISTS (
    SELECT 1
    FROM `transaction` t
    WHERE t.company_id = c.id
);
```

Resultado:

Result Grid | Filter Rows:

company_name

Output:

Output

Action Output

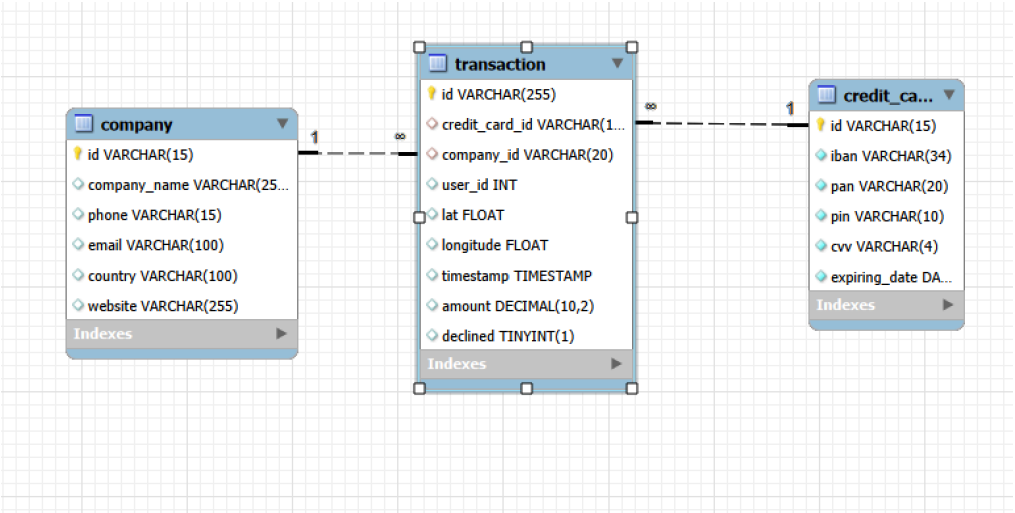
#	Time	Action	Message	Duration / Fetch
100138	19:35:28	SELECT c.company_name FROM company c WHERE c.id NOT IN (SELECT ...	0 row(s) returned	0.031 sec / 0.000 sec

EXERCICI 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest

Crear tabla, la columna expiring_date estaba en la tabla original con un formato no conveniente por lo que al crear la tabla ese campo lo designe con formato VARCHAR para después decirle que era mes, día y año. Por último, cambié el formato de VARCHAR a DATE.

```sql
CREATE TABLE credit_card (
    id VARCHAR(60) PRIMARY KEY,
    iban VARCHAR(60)NOT NULL,
    pan VARCHAR(60) NOT NULL,
    pin VARCHAR(60) NOT NULL,
    cvv VARCHAR(60) NOT NULL,
    expiring_date VARCHAR(60) NOT NULL
);

UPDATE credit_card
SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');

ALTER TABLE credit_card
MODIFY expiring_date DATE;

ALTER TABLE `transaction`
ADD FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);
```



El diagrama representa un modelo relacional donde la tabla transaction hace de nexo entre la tabla company y la tabla credit_card. La relación que hay entre company/transaction es de 1:N, una empresa puede tener muchas operaciones, una operación es realizada por una única empresa y la relación que hay entre credit_card/transaction también es de 1:N, una tarjeta puede tener muchas operaciones.

La tabla transaction permite relacionar qué tarjeta se usó, qué empresa la usó, cuándo y cuánto fué el importe de la operación. Esta relación se da lugar por las claves foráneas.

EXERCICI 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR3234563122135768176999999. Recorda mostrar que el canvi es va realitzar.

Se actualiza el campo iban del ID 'CcU-2938' mediante un UPDATE y se le asigna nuevo valor, después se realiza un SELECT para verificar que se ha realizado el cambio.

Previo. Comprobamos que ese id existe.

```
SELECT *
FROM credit_card
WHERE id = 'CcU-2938';
```

Resultado:

	id	iban	pan	pin	cvv	expiring_date
▶	CcU-2938	TR3019503122135768176386 61	5424465566813633	3257	984	2022-10-30

Output:

✔	9	11:16:26	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned

Consulta. Hemos comprobado que existe, por lo que, actualizamos el iban del id y hacemos comprobación:

```
109 •  UPDATE credit_card
110    SET iban = 'TR3234563122135768176999 99'
111    WHERE id = 'CcU-2938';
112
113 •  SELECT *
114    FROM credit_card
115    WHERE id = 'CcU-2938';
```

Resultado:

Result Grid | Filter Rows: | Edit: | Export/Import:

	id	iban	pan	pin	cvv	expiring_date
▶	CcU-2938	TR3234563122135768176999 99	5424465566813633	3257	984	2022-10-30

Output:

	#	Time	Action	Message
✔	10	11:19:41	UPDATE credit_card SET iban = 'TR3234563122135768176999 99' WHERE id = '...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✔	11	11:20:09	SELECT * FROM credit_card WHERE id = 'CcU-2938'	1 row(s) returned

EXERCICI 6
En la taula "transaction" ingressa una nova transacció amb la següent informació:

Para agregar la nueva fila antes he tenido que agregar los id en credit_card y company ya que ambos son FK y es obligado que no estén vacíos esos campos. Además he modificado la tabla original de credit_card porque una de sus características es que alguno de los campos no aceptaban nulos por lo que tuve que cambiarlo a que sí aceptasen.

Previo:

```
INSERT INTO company (id)
VALUES (
    'b-9999');

ALTER TABLE credit_card
MODIFY iban VARCHAR(34) NULL,
MODIFY pan VARCHAR(20) NULL,
MODIFY pin VARCHAR(10) NULL,
MODIFY cvv VARCHAR(4) NULL,
MODIFY expiring_date DATE NULL;

INSERT INTO credit_card(id)
VALUES ('CcU-9999');
```

Output de agregación de nuevo id en tabla company:

```
13  11:22:28  INSERT INTO company (id) VALUES (  'b-9999')        1 row(s) affected
```

Consulta:

```
INSERT INTO `transaction`(id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)
VALUES (
    '108B1D1D-5B23-A76C-55EF-C568E49A99DD',
    'CcU-9999',
    'b-9999',
    '9999',
    829.999,
    -117.999 ,
    NULL,
    111.11,
    0
)
```

Resultado:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	0

Output:

Output

Action Output

#	Time	Action	Message	Duration / Fetch
14	10:21:09	INSERT INTO `transaction`(id, credit_card_id, company_id, user_id, lat, longitu...	1 row(s) affected	0.078 sec

EXERCICI 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

Para eliminar la columna he usado ALTER TABLE con el nombre de la tabla y el comando DROP column con el nombre de la columna a eliminar.
He hecho un SELECT para mostrar los cambios.

Consulta:

```
ALTER TABLE credit_card DROP column pan;

SELECT *
FROM credit_card;
```

Resultado:

Result Grid | Filter Rows: | Edit: | Export/Import:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	2025-09-27
CcS-4858	XX8581768137002436094025	3964	817	2028-12-28
CcS-4859	XX7826930491423553609370	4983	277	2026-11-26
CcS-4860	XX5559590368835304645299	6876	661	2027-07-27
CcS-4861	XX2035182877195191627307	5710	398	2026-04-25

Output:

Output

Action Output

#	Time	Action	Message
21	10:30:01	SELECT * FROM credit_card	5001 row(s) returned

EXERCICI 8

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

El diagrama final sería el de la foto, para llegar hasta ahí:

- Primero he creado la base de datos y después las tablas
- Tras crear las tablas he importado los datos, algunas tablas con Wizard y otras con una consulta porque era más rápido y no me daba error.
- He corregido algunos errores con las fechas y sus formatos.
- A la hora de importar los registros de las tablas de usuarios americanos y europeos he dado un valor a la columna 'region' predeterminado para todos los registros de cada tabla respectivamente.

:

```sql
CREATE DATABASE transactions2;
USE transactions2;
```

```sql
CREATE TABLE transactions (
    id VARCHAR(255) PRIMARY KEY,
    card_id VARCHAR(15),
    business_id VARCHAR(20),
    timestamp TIMESTAMP,
    amount DECIMAL(10,2),
    declined TINYINT(1),
    product_ids VARCHAR(255),
    user_id VARCHAR(15),
    lat FLOAT,
    longitude FLOAT
);
```

```sql
CREATE TABLE companies (
    company_id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(20),
    email VARCHAR(255),
    country VARCHAR(100),
    website VARCHAR(255)
);
```

```sql
CREATE TABLE credit_cards (
    id VARCHAR(15) PRIMARY KEY,
    user_id VARCHAR(15),
    iban VARCHAR(34),
    pan VARCHAR(20),
    pin VARCHAR(4),
    cvv VARCHAR(4),
    track1 VARCHAR(255),
    track2 VARCHAR(255),
    expiring_date DATE
);
```

```sql
CREATE TABLE users (
    id VARCHAR(15) PRIMARY KEY,
    name VARCHAR(25),
    surname VARCHAR(25),
    phone VARCHAR(20),
    email VARCHAR(255),
    birth_date DATE,
    country VARCHAR(100),
    city VARCHAR(25),
    postal_code VARCHAR(10),
    address VARCHAR(100),
    Region VARCHAR(100)
);
```

Agrego los registros de las tablas:

```sql
LOAD DATA LOCAL INFILE "C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ESPECIALIDAD/EJERCICICIOS/SPRINT 1/N1.Ex.8__ transactions.csv"
INTO TABLE transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.1999	1.43554
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.5972	12.2218
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.7573	-95.3796
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.2084	5.69081
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19	215	53.5535	-113.499
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2023-12-31 00:06:36	383.63	0	93, 55, 28, 91	2928	51.9362	5.34265
0009D494-6245-4DE9-955D-2C084191CFEB	CcS-8483	b-2526	2017-07-18 07:52:02	197.80	0	55, 8, 72	3902	45.492	-73.5706

transactions 1

```sql
LOAD DATA LOCAL INFILE 'C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ESPECIALIDAD/EJERCICICIOS/SPRINT 1/N1.Ex.8__ companies.csv'
INTO TABLE companies
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

company_id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one
b-2250	Amet Nulla Donec Corporation	07 15 25 14 74	mattis.integer.eu@protonmail.net	Italy	https://netflix.com/sub/cars

```
LOAD DATA LOCAL INFILE 'C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ESPECIALIDAD/EJERCICICIOS/SPRINT 1/N1.Ex.8__ credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(id, user_id, iban, pan, pin, cvv, track1, track2, @exp_date)
SET expiring_date = STR_TO_DATE(@exp_date, '%m/%d/%y');
```

id	user_id	iban	pan	pin	cvv	track1	track2	expiring_da
CcS-4857	276	XX4857591835292505850771	2314242385113924	1819	467	%B2314242385113924^LWCBUDLWCBUD^22...	%B2314242385113924=2410101518363164?	2025-09-27
CcS-4858	277	XX8581768137002436094025	6582720299715533	3964	817	%B6582720299715533^TIQMVITIQMVI^2404...	%B6582720299715533=2411101104546272?	2028-12-28
CcS-4859	278	XX7826930491423553609370	8861684536289642	4983	277	%B8861684536289642^COFBGDCOFBGD^280...	%B8861684536289642=2502101761665371?	2026-11-26
CcS-4860	279	XX5559590368835304645299	2481155515498459	6876	661	%B2481155515498459^TIUJTUTIUJTU^31040...	%B2481155515498459=2602101514414395?	2027-07-27
CcS-4861	280	XX2035182877195191627307	1308930301149557	5710	398	%B1308930301149557^HPOBNZHPOBNZ^330...	%B1308930301149557=2805101751305028?	2026-04-25

Output:

- Crear tabla transactions y agregar registros:

3 11:31:25 CREATE TABLE transactions (id VARCHAR(255) PRIMARY KEY, card_id VAR... 0 row(s) affected, 1 warning(s): 1681 Integer display width is deprecated and will be re...

10 11:35:04 LOAD DATA LOCAL INFILE "C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ES... 100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

- Crear tabla companies y agregar registros:

5 11:32:39 CREATE TABLE companies (company_id VARCHAR(15) PRIMARY KEY, co... 0 row(s) affected

12 11:36:19 LOAD DATA LOCAL INFILE 'C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ES... 100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

- Crear tabla credit_cards y agregar registros:

7 11:33:22 CREATE TABLE credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VAR... 0 row(s) affected

14 11:37:09 LOAD DATA LOCAL INFILE 'C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ES... 5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

- Crear tabla users:

9 11:34:37 CREATE TABLE users (id VARCHAR(15) PRIMARY KEY, name VARCHAR(25), ... 0 row(s) affected

Importo los registros de las tablas american_users y european_users a la tabla users.

```sql
LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.4/Uploads/N1-Ex.8__ american_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
SET region = 'America';

LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.4/Uploads/N1.Ex.8__ european_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
SET region = 'Europa';
```

Output:

#	Time	Action	Message
27	12:06:06	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.4/Uploads/...	
28	12:06:14	LOAD DATA LOCAL INFILE 'C:/ProgramData/MySQL/MySQL Server 8.4/Uploads/...	

Comprobamos que la importación se hizo correctamente.

```sql
SELECT * FROM transactions2.users;
```

Resultado:

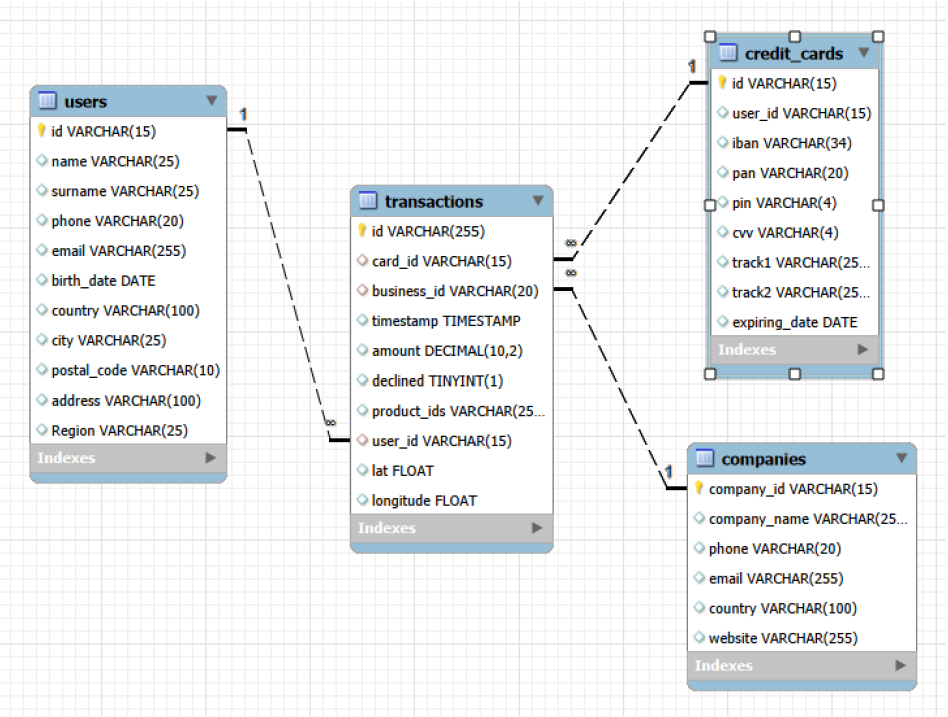
id	name	surname	phone	email	birth_date	country	city	postal_code	address	Region
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	0000-00-00	United States	New York	10001	348-7818 Sagittis St.	America
10	Robert	Mccarthy	(324) 746-6771	fermentum@protonmail.com	0000-00-00	United States	San Jose	95101	P.O. Box 773	America
100	Melodie	Mclean	1-677-221-7152	risus.varius@google.ca	0000-00-00	United States	San Jose	95101	Ap #644-8492 Sagittis St.	America
1000	Amkjrv	Qbulrxbp	+48-258-9936	amkjrv.qbulrxbp@example.com	0000-00-00	Germany	Stuttgart	70173	215 Qbulrxbp St	Europa
1001	Nfvrlb	Oydaiwbg	+94-121-2522	nfvrlb.oydaiwbg@example.com	0000-00-00	Germany	Cologne	50667	121 Oydaiwbg St	Europa

Output:

29	12:07:24	SELECT * FROM transactions2.users	5000 row(s) returned

Para agregar las FK a las tablas creo una consulta con el comando ALTER TABLE donde agrego la restricción (ADD CONSTRAINT) para cada una de las FK.

```sql
ALTER TABLE transactions
ADD CONSTRAINT fk_business
    FOREIGN KEY (business_id) REFERENCES companies(company_id),
ADD CONSTRAINT fk_card
    FOREIGN KEY (card_id) REFERENCES credit_cards(id),
ADD CONSTRAINT fk_user
    FOREIGN KEY (user_id) REFERENCES users(id);
```



EXERCICI 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

He creado una subconsulta que agrupa a usuarios y filtra aquellos que tienen más de 80 operaciones. En la consulta principal el WHERE da solo esos usuarios.

Consulta:

```
SELECT *
FROM users u
WHERE EXISTS (
    SELECT 1
    FROM transactions t
    WHERE t.user_id = u.id
    GROUP BY t.user_id
    HAVING COUNT(*) > 80
);
```

Resultado:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	Region
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	0000-00-00	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi Rd.	Europa
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	0000-00-00	Germany	Stuttgart	70173	82 Hwcru Street	Europa
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	0000-00-00	Italy	Genoa	16100	53 Astuw Street	Europa
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	0000-00-00	Poland	Gdansk	80-001	52 Xgvfridxs Street	Europa

Output:

20	11:47:19	SELECT * FROM users u WHERE EXISTS (SELECT 1 FROM transactions t ...	4 row(s) returned	

EXERCICI 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Para mostrar el resultado he hecho dos JOINS para agregar las tablas transactions y credit_cards a la consulta con companies. He filtrado con el WHERE con el nombre de la compañía y he agrupado por id de la empresa, nombre y por último iban.

Consulta:

```
SELECT c.company_id, c.company_name, cc.iban, ROUND(AVG(t.amount),2) AS media_importe
FROM companies c
JOIN transactions t ON t.business_id = c.company_id
JOIN credit_cards cc ON cc.id = t.card_id
WHERE c.company_name = 'Donec Ltd'
GROUP BY c.company_id, c.company_name, cc.iban
ORDER BY media_importe DESC;
```

Resultado:

company_id	company_name	iban	media_importe
b-2242	Donec Ltd	XX3830178139196201993663 52	680.69
b-2242	Donec Ltd	XX6377063573975703949739 13	680.01
b-2242	Donec Ltd	XX9713939714652922023122 59	645.46
b-2242	Donec Ltd	XX1718471169288923759693 07	628.89
b-2242	Donec Ltd	XX2254246388185424062235 75	608.68
b-2242	Donec Ltd	XX7488907290571957117660 71	607.29
b-2242	Donec Ltd	TN9614563570667381893122	605.41
b-2242	Donec Ltd	XX4819080340373642425911 85	605.36
b-2242	Donec Ltd	XX1946755197392563357535 08	597.19
b-2242	Donec Ltd	XX2159627660619671954934 37	594.26
b-2242	Donec Ltd	[illegible]	[illegible]

Output:

19	12:44:08	SELECT c.company_id, c.company_name, cc.iban, ROUND(AVG(t.amount),2) AS ...	371 row(s) returned

NIVEL 2

EXERCICI 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vende.

He creado una consulta donde muestra fecha, la suma del importe de las operaciones, agrupadas por día, ordenador por esta cifra y limitando la visualización a 5 filas.

Consulta:

```
SELECT DATE(t.timestamp), SUM(t.amount) AS total_importe
FROM transactions t
GROUP BY DATE(t.timestamp)
ORDER BY total_importe DESC
LIMIT 5;
```

Resultado:

	DATE(t.timestamp)	total_importe
▶	2022-12-13	14337.44
	2019-11-18	13591.32
	2023-02-20	13332.59
	2017-12-20	13318.43
	2019-03-18	12680.95

Output:

✔	20	12:45:03	SELECT DATE(t.timestamp), SUM(t.amount) AS total_importe FROM transactions t ...	5 row(s) returned

EXERCICI 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat

Para llegar al resultado parto de la tabla companies, hago JOIN con transactions para poder filtrar con WHERE entre las dos cantidades, además filtré por las tres fechas, por último ordené por la columna amount en manera descendente.

Consulta:

```
SELECT c.company_name, c.phone,c.country, DATE(t.timestamp) AS Fecha, t.amount
FROM companies c
JOIN transactions t ON t.business_id = c.company_id
WHERE t.amount BETWEEN 350 AND 400
AND DATE(t.timestamp) IN ('2015-04-29','2018-07-20', '2024-03-13')
ORDER BY t.amount DESC;
```

Resultado:

company_name	phone	country	Fecha	amount
Aliquam PC	01 45 73 52 16	Germany	2024-03-13	399.84
Auctor Mauris Vel LLP	08 09 28 74 14	United States	2018-07-20	399.51
At Pede Corp.	06 14 48 33 15	Italy	2015-04-29	390.69
Aliquam PC	01 45 73 52 16	Germany	2024-03-13	388.29
Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	2018-07-20	373.71
Fringilla LLC	08 29 15 93 57	New Zealand	2015-04-29	367.62
Pede Cum Ltd	07 62 26 48 38	Norway	2018-07-20	356.87
Auctor Mauris Vel LLP	08 09 28 74 14	United States	2024-03-13	353.75

Output:

21 12:45:41 SELECT c.company_name, c.phone,c.country, DATE(t.timestamp) AS Fecha, t.amo... 8 row(s) returned

EXERCICI 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen més de 400 transaccions o menys.

Para realizar el ejercicio creo una consulta donde cuento todos los id de la tabla transaction. Creo una condicional donde le indico que si ese conteo es mayor de 400 entonces que ponga como registro en la columna nueva 'Alta' de lo contrario 'Baja'.
Agrupo por company_id de la tabla companies.

Consulta:

```
SELECT
    c.company_id,
    c.company_name,
    COUNT(t.id) AS total_op,
    CASE
        WHEN COUNT(t.id) > 400 THEN 'Alta'
        ELSE 'Baja'
    END AS clasificacion
FROM companies c
JOIN transactions t ON c.company_id = t.business_id
GROUP BY c.company_id
ORDER BY total_op DESC;
```

Resultado:

company_id	company_name	total_op	clasificacion
b-2222	Ac Fermentum Incorporated	2401	Alta
b-2302	Nunc Interdum Incorporated	1599	Alta
b-2330	Donec Fringilla PC	1593	Alta
b-2366	Mauris Institute	1586	Alta
b-2614	Rutrum Non Inc.	1585	Alta
b-2350	Aliquet Vel Vulputate Incorporated	1583	Alta

Output:

7 10:35:04 SELECT c.id, c.company_name FROM company c WHERE EXISTS (SELECT 1 F... 100 row(s) returned

EXERCICI 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

Consulta con el comando DELETE dentro de la tabla 'transaction' usando el filtro WHERE.

Output:

```
DELETE FROM transactions
WHERE id ='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';

SELECT *
FROM transactions
WHERE id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Resultado:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Output:

23 12:47:26 DELETE FROM transactions WHERE id ='000447FE-B650-4DCF-85DE-C7ED0EE1... 1 row(s) affected

EXERCICI 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

Consulta:

```
CREATE OR REPLACE VIEW VistaMarketing AS
SELECT c.company_name, c.phone, c.country, ROUND(AVG(t.amount),2) AS media_compra
FROM companies c
JOIN transactions t ON t.business_id = c.company_id
GROUP BY c.company_name, c.phone, c.country;
```

```
SELECT * FROM VistaMarketing
ORDER BY media_compra DESC;
```

Resultado:

company_name	phone	country	media_compra
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.87
Pretium Neque Corp.	07 77 48 55 28	Australia	276.16
Urna Convallis Associates	06 01 24 77 04	United States	274.24
At Associates	09 56 61 10 65	New Zealand	272.21
Metus Vitae Associates	08 25 44 40 66	Australia	270.08
Aliquet Diam Limited	02 76 61 47 46	United States	269.60
Nec Luctus LLC	02 14 71 75 73	Norway	268.60
Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.85
Tortor Nunc Commodo Company	05 35 92 77 16	United States	267.84
Cras Consulting	07 50 10 85 63	Belgium	267.44

Output:

35 12:54:24 SELECT * FROM VistaMarketing ORDER BY media_compra DESC 100 row(s) returned

NIVEL 3

EXERCICI 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

👉 *Quantes targetes estan actives?*

Para crear la tabla he hecho:

- Un subconsulta donde uso el comando ROW_NUMBER con el cual da un número y donde por cada credit_card_id ordena por fecha de la operación, la más reciente a la más lejana, en la fecha más reciente crea una columna con un 1, la fecha siguiente usaría el 2 y así sucesivamente..
- En la consulta principal mostramos el id de las tarjetas, un condicional con la función CASE, en la cual suma cuando la operación ha sido rechazada(declined = 1), si el valores de esa suma es menor de tres entonces, la tarjeta tiene una clasificación de 'Activa', en caso contrario, 'Inactiva'.
- Por último, una consulta para comprobar cuántas tarjetas son Activas e Inactivas.

Consulta:

```sql
CREATE TABLE estado_tarjeta AS
SELECT card_id,
    CASE
        WHEN SUM(declined) <3 THEN 'Active'
        ELSE 'Inactive'
    END AS estado
FROM (SELECT  card_id, timestamp, declined,
            ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as orden_operaciones
            FROM transactions) t
WHERE orden_operaciones <= 3
GROUP BY card_id;
```

```sql
SELECT Estado, COUNT(*)
FROM estado_tarjeta
GROUP BY Estado;
```

Resultado:

	Estado	COUNT(*)
▶	Active	4995
	Inactive	5

Output:

✔	40	12:57:44	SELECT Estado, COUNT(*) FROM estado_tarjeta GROUP BY Estado	2 row(s) returned

EXERCICI 2

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

👉 *Necessitem conèixer el nombre de vegades que s'ha venut cada producte.*

Para conseguir el resultado final he seguido lo siguientes pasos:

- Primero he creado la tabla products.
- Después he importado el csv con todos los datos. Como el formato de la columna price estaba erróneo (incluía el símbolo $ y estaba definida como texto), en el mismo LOAD DATA LOCAL INFILE, reemplacé el símbolo del dólar con un CASE donde he puesto que si hay algún registro vacío que sea formato NULL y sino, que haga el reemplazo del símbolo del $.
- Para resolver el ejercicio he usado la función JSON_TABLE para separar los identificadores de productos. De esta manera, cada producto pasa a ocupar una fila independiente. Dentro de JSON_TABLE, los ids de la columna 'product_ids' los he unido con un corchete al principio y al final para que se convirtiera en una estructura de datos.
- Con este resultado, en la misma consulta he realizado un COUNT agrupado por product_id para obtener el número de veces que se ha vendido cada producto.
-

Consulta:

```sql
CREATE TABLE products (
    id VARCHAR(15) PRIMARY KEY,
    product_name VARCHAR(255),
    price DECIMAL(10,2),
    colour VARCHAR(20),
    weight FLOAT,
    warehouse_id VARCHAR(10)
);
```

```sql
LOAD DATA LOCAL INFILE 'C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ESPECIALIDAD/EJERCICICIOS/SPRINT 1/N3.Ex.2__ products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(@id, @product_name, @price, @colour, @weight, @warehouse_id)
SET
    id = @id,
    product_name = @product_name,
    price =CASE
        WHEN @price = '' THEN NULL
        ELSE CAST(REPLACE(@price, '$', '') AS DECIMAL(10,2))
    END,
    colour = @colour,
    weight = @weight,
    warehouse_id = @warehouse_id;
```

#	Time	Action	Message
75	17:58:36	CREATE TABLE products (id VARCHAR(15) PRIMARY KEY, product_name VA...	0 row(s) affected
76	17:58:40	LOAD DATA LOCAL INFILE 'C:/Users/herre/OneDrive/Desktop/IT ACADEMY/ES...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

```sql
CREATE TABLE op_products AS
SELECT
    t.id AS transaction_id,
    pi.product_id
FROM transactions t,
JSON_TABLE(
    CONCAT('[',t.product_ids, ']'),
    '$[*]'
    COLUMNS (product_id VARCHAR(5) PATH '$')
) pi;
```

```sql
SELECT product_id, COUNT(product_id) AS total_vendido
FROM op_products
GROUP BY product_id;
```

Resultado:

product_id	total_vendido
16	2608
26	2536
97	2536
87	2598
66	2601
69	2457
30	2468
11	2573
81	2469

Output:

#	Time	Action	Message
77	18:00:36	CREATE TABLE op_products AS SELECT t.id AS transaction_id, pi.product_i...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0
78	18:00:42	SELECT product_id, COUNT(product_id) AS total_vendido FROM op_products GR...	100 row(s) returned